Component Decomposition for the TCTL Tableau

Which Half of the Blow-Up Is Avoidable, and How

How to read this

Two audiences. Someone deciding whether the idea is worth a paper should read Section 1 (what the measurement says the problem actually is), Section 2 (the theorem), and Section 4 (what it buys). Someone maintaining the code should read Section 3 and keep Section 5 nearby, because most of what looks like an optimisation here is not one, and that section says which.

Every mechanism is traceable to one place in the source.

Mechanism	Where	Section
partition a node into components	`components()`	Section 3
decide a node by its components	`decide_by_components()`	Section 3
hook into the search	`try_split` in `decide_scc`	Section 3
atom sets as bitmasks	`atoms_of()`	Section 3
clock ownership	`clocks_of()`	Section 3
shapes that never split	`nosplit_`	Section 3
root-level split (static form)	`Tableau::check`	Section 4

Both forms are behind flags and default off: `TCTL_COMPONENTS=1` for the per-node form, `TCTL_DECOMPOSE=1` for the root-level one. With neither set the search is exactly what it was.

In plain words. The tableau's cost has two completely different causes, and they need different cures. One is that a node carries many clock regions; the other is that it carries many independent obligations. Everything anyone would normally try attacks the first. The measurement in Section 1 says the expensive cases are the second, and the second has a clean fix.

---

1 The problem, separated by measurement

A tableau node is $\langle \Gamma, \mathcal{X}, Z, \mathcal{O}, \mathcal{D} \rangle$ together with the pursuits live at it. Its cost has two independent causes, and the whole point of this document is that they are not the same problem.

1.1 Two regimes

Count, for a run, the number of *distinct discrete shapes* — nodes taken up to their zone and their edge markers — against the number of nodes.

instance	shapes	nodes	zones per shape
16 conjoined $\mathrm{EF}_{[a_j,b_j]}\, p_j$	262 204	263 162	**1.00**
depth-7 $\mathrm{AG}(\mathrm{EF}_{[0,2]} \cdots)$ tower	5 841	10 785	1.85
4-level tower, varied windows	2 635	17 370	6.59

The first row is the important one. On the most expensive family in the suite the zone contributes *nothing*: there is one zone per discrete shape, and the quarter of a million nodes are a quarter of a million distinct *discrete* states. No abstraction, no widening and no inclusion test can touch that number, because there is nothing to abstract.

1.2 What those discrete states are

Instrumenting the pursuit vector on the depth-6 tower gives a maximum of 12 simultaneous pursuits, exactly two per nesting level, with no modality duplicated. The histogram of $|P|$ over the nodes is $5, 22, 77, 172, 323, 545, 824, 952, 903, 803, 485, 167, 25$: binomial, peaked in the middle. That is the signature of a search ranging over the *subsets* of a set of independent obligations.

For $\bigwedge_{j<k} \mathrm{EF}_{[a_j,b_j]}\, p_j$ the reading is direct. Each conjunct is an eventuality that is either still outstanding or already witnessed, the conjuncts do not interact, and the node records the whole vector of k bits. The search visits 2^k of them, and measured node counts follow: 1 247, 16 863, 66 179, 262 975, 1 049 619 for $k = 8, 12, 14, 16, 18$.

1.3 There is nothing to learn from, either

The obvious import from SAT and SMT is conflict-driven search: learn why a branch died and prune its siblings. Counting how each node ends says that this has no purchase here.

instance	accepted	rejected	closed
16 conjoined eventualities (SAT)	262 975	**0**	**0**
4-level tower (SAT)	12 928	1 014	790
depth-6 tower (SAT)	4 281	609	385
$\mathrm{AG}(\mathrm{EF}_{[3,4]}p) \wedge \mathrm{AG}_{[4,5]}\neg p$ (UNSAT)	161	108	53

On the hardest instance every single node is accepted. The search is not thrashing through contradictory subtrees that a learned clause would have cut; it is enumerating states that are all genuinely consistent. This is the structural difference from DPLL, where clause learning pays precisely because the search spends most of its life in conflict.

In plain words. Asking a SAT solver's tricks to help here is asking a technique for *failing faster* to speed up a search that never fails. What is needed is a technique for not enumerating a product, and that one does exist in the same literature — component caching, from model counting.

1.4 The claim

The 2^k is an artefact of representation, not of the problem. The k conjuncts share no atom and no clock; a model for the conjunction can be assembled from a model for each. Section 2 makes that precise and proves it, Section 3 turns it into a test cheap enough to run at every node, and Section 4 reports that the 2^{18} becomes 1 249.

2 The decomposition theorem

Write $\mathrm{at}(\varphi)$ for the atomic propositions occurring in φ and $\mathrm{cl}(\varphi)$ for the formula-clocks of the timed modalities occurring in it. A node's clocks are allocated per modality, so $\mathrm{at}(\varphi) \cap \mathrm{at}(\psi) = \emptyset$ for every pair drawn from different components already gives $\mathrm{cl}(\cdot)$ disjoint as well; the two are not independent assumptions.

Definition 2.1 (Split)**.** A node $N = \langle \Gamma, \mathcal{X}, Z, \mathcal{O}, \mathcal{D} \rangle$ with pursuit set P *splits* into $N_1, \ldots, N_r$ ($r \geq 2$) when

(S1) Γ, $\mathcal{X}$ and P are partitioned among the N_i, and $\mathrm{at}(N_i) \cap \mathrm{at}(N_j) = \emptyset$ for $i \neq j$;

(S2) every clock constrained in Z belongs to exactly one $\mathrm{cl}(N_i)$;

and N_i carries $Z{\downarrow}\mathrm{cl}(N_i)$, the deadlines on its own clocks, and the phase of N.

There is deliberately no condition on the zone. An earlier version required Z to be the product of its projections, $Z = \bigcap_i Z{\downarrow}\mathrm{cl}(N_i)$, on the grounds that a reset leaves finite difference constraints in the DBM and can relate x and y even when the modalities owning them share no atom. That condition is sufficient but *not* necessary, and the proof below never uses it: see Remark 2.3. (S2) is a real requirement, and it is about a clock owned by no group at all: a modality still sitting in Γ, not yet launched, already owns its clock, and that clock may already carry a value.

Theorem 2.2 (Decomposition)**.** *If N splits into $N_1, \ldots, N_r$, then N is realisable if and only if every N_i is.*

Proof. ($\Rightarrow$) Let $\mathcal{T}$ realise N, with location s and the interpretation of the formula-clocks fixed by $\mathcal{O}$. Fix i and let $\mathcal{T}_i$ be $\mathcal{T}$ with its labelling restricted to $\mathrm{at}(N_i)$ and its clocks restricted to $\mathrm{cl}(N_i)$. Every formula of N_i mentions only atoms in $\mathrm{at}(N_i)$ and is evaluated against clocks in $\mathrm{cl}(N_i)$, and the satisfaction relation of TCTL inspects a configuration only through those two, so truth of each such formula is unchanged by the restriction; the same runs remain, with the same delays, so Runs^∞ is unchanged and the quantifiers range over the same set. Every $\nu \in Z{\downarrow}\mathrm{cl}(N_i)$ extends, by definition of the projection, to some $\nu' \in Z$, and $\mathcal{T}, (s, \nu')$ satisfies the formulas of N_i, hence so does $\mathcal{T}_i, (s, \nu)$. Thus $\mathcal{T}_i$ realises N_i.

($\Leftarrow$) Let $\mathcal{T}_i$ realise N_i at (s_i, ν_i), induced by a TA $\mathcal{A}_i$ over clock set $\mathrm{cl}(N_i)$. The clock sets are pairwise disjoint, so form the product $\mathcal{A} = \prod_i \mathcal{A}_i$ in the standard way: locations are tuples, the invariant of a tuple is the conjunction of the components' invariants, the labelling is the union of the components' labellings — consistent, because the atom sets are disjoint — and the edges interleave, each edge of one $\mathcal{A}_i$ lifting to an edge of the product that leaves the other components' locations and clocks untouched. Delays synchronise: a delay of t in the product is a delay of t in every component, admissible because each component's invariant permits it.

Three things now hold of $\mathcal{A}$.

Projection of runs. Every run ρ of $\mathcal{A}$ projects to a run $\rho{\downarrow}i$ of $\mathcal{A}_i$ with the same delays, and ρ is non-Zeno iff its projections are, the delays being shared. Conversely, given a run ρ_i of $\mathcal{A}_i$ and any runs of the others, the delays can be merged into a single run of the product projecting to each: refine the components' delay sequences to their common refinement and interleave the discrete steps in any order consistent with time. Non-blockingness of each component gives non-blockingness of the product, so Runs^∞ is non-empty everywhere.

Truth is componentwise. By induction on φ with $\mathrm{at}(\varphi) \subseteq \mathrm{at}(N_i)$: for a literal this is the definition of the product labelling; the boolean cases are immediate; for $\mathrm{E}(\cdot)$ take the witnessing run of $\mathcal{A}_i$ and lift it against arbitrary runs of the others, and for $\mathrm{A}(\cdot)$ take any run of the product and project it to $\mathcal{A}_i$, where the hypothesis applies. The timed operators add only a constraint on $\mathit{time}(\pi)$, which the projection preserves because delays are shared.

The zone. Let $\nu \in Z$. By (S2) each of its coordinates belongs to exactly one component, so ν decomposes as $(\nu_1, \ldots, \nu_r)$ with $\nu_i \in Z{\downarrow}\mathrm{cl}(N_i)$, and (s_i, ν_i) realises N_i's formulas. By the previous paragraph the product configuration $((s_1, \ldots, s_r), \nu)$ satisfies every formula of every N_i, that is, all of $\Gamma \cup \mathcal{X}$. As $\nu \in Z$ was arbitrary, $\mathcal{A}$ realises N. □

Remark 2.3 (No zone-product condition is needed)*.* Take Z constrained by $x - y < 1$ with x, y in different components: both projections are universal and Z is not, so Z is a proper

subset of the product of its projections. Splitting is still sound, and each direction says why. Forwards, a valuation of a projection extends to one of Z, and a component's formulas read only its own clocks. Backwards, each component is asked to be realisable over its *projection*, which is the larger demand: $Z \subseteq \bigcap_i Z{\downarrow}\mathrm{cl}(N_i)$, so a product model meeting the projections meets Z a fortiori.

Dropping the condition is a simplification of the proof rather than a measured gain: on every instance in the suite it never once blocked a split that (S1) and (S2) allowed, and node counts are identical with it and without it.

Corollary 2.4 (What the search may do)**.** *If N splits, its verdict in the tableau is the conjunction of the verdicts of its components.*

Proof. The calculus is sound and complete, so a node lies in the greatest successful marking exactly when it is realisable; apply Theorem 2.2. □

In plain words. Two halves of a node that never mention the same symbol and never share a clock are two separate questions, even if the zone happens to relate their clocks — neither half can read the other's clock, so the relation is invisible to both. Answer the two questions separately and combine the answers by running the two models side by side: they agree about the passage of time and about nothing else, which is exactly what a product automaton is.

---

3 The algorithm

3.1 Partitioning a node

`components()` treats each goal of Γ, each context formula of $\mathcal{X}$ and each live pursuit as one *item*, and runs union–find over the items, joining two whenever their symbol sets meet. That is (S1).

Atom sets are **bitmasks**, one bit per atom of the input, memoised per formula id. The test compares every pair of items at every node, so as sets of strings it is three nested loops with a string compare at the bottom, and it measured about a third of the run on formulas that never split. As words it is one `&`. An input with more than 64 atoms simply does not decompose.

3.2 The side conditions, and where each bites

Clock ownership ((S2)). Ownership is read off the *formulas*, not off the live pursuits, via `clocks_of()`: a modality still sitting in Γ already owns its formula-clock. Deriving ownership from pursuits alone leaves such a clock owned by nobody, and the component construction then frees it out of *every* component's zone. That is a weakening, and it is the exact bug that made $\mathrm{AG}(\mathrm{EF}_{[2,4]}p) \wedge \mathrm{AG}_{[4,7]}\neg p$ come back satisfiable: freeing the onset clock erases the constraint that distinguishes one onset from another. A clock owned by more than one group, or constrained but owned by none, aborts the split.

Shared symbols are not only propositions. A data variable couples its constraints exactly as an atom does: $\{x > 3\}$ and $\{x < 2\}$ mention no proposition at all, and grouping by propositions alone puts them in different components, each satisfiable on its own. `IntVar` names therefore join the bitmask alongside `Atom` names. The self-test caught this on twelve checks of the form $\{x \leq 5\} \wedge \{x \geq 10\}$; the sweeps did not, having no data variables in them.

Degenerate components. A component mentioning no atom at all — a bare $\top$ or $\bot$ — is folded into the first real component instead of being split off. Peeling those off is all cost and no benefit, and empirically it was also where the first version went wrong: every split it performed on $\mathrm{AG}(\mathrm{EF}_{[2,4]}p) \wedge \mathrm{AG}_{[4,7]}\neg p$ was degenerate, and the verdict was still wrong, which is what localised the fault to clock ownership rather than to the partition.

3.3 Deciding by components

`decide_by_components()` keys each component on its node key and looks it up in a cache shared with every sub-search. On a miss it runs a fresh engine from that node — constructed from the original root formula, so the clock allocation, the maximum constants and the split points are identical — and stores the verdict. One false component is enough to reject.

The cache is what makes this component *caching* rather than component *splitting*: the same component recurs across many parent nodes, which is precisely why the undecomposed search enumerated their product.

3.4 Hooking it into the search

`try_split` runs in `decide_scc` before a node is first visited. A node that splits is marked settled with the conjunction of its components' verdicts and is never expanded: it becomes a leaf of the enclosing search, exactly like a node whose own SCC has already closed, which the search already knows how to consume. Tarjan never sees it, so no cycle can run through it — which is sound because Corollary 2.4 decides it outright.

Recursion terminates because every component is strictly smaller than the node it came from; a depth cap guards against pathology.

3.5 Not re-deciding shapes that never split

Whether a node splits depends only on *which formulas it carries*, never on its zone. So a shape that did not split never will, and `nosplit_` records its signature. Shapes repeat heavily — a nested tower has roughly half as many shapes as nodes — and without the memo the union–find runs again at each of them, which is pure loss on any formula that never decomposes. This is what brings the overhead on non-splitting inputs down to a few per cent (Section 4).

3.6 Two closures that were buying nothing

Two costs found by re-profiling after the decomposition work, both unrelated to it and both removed on a proof rather than a measurement.

The dead-clock closure. Eliminating the clocks a node no longer reads used to re-close the DBM, which was 30% of the run. It is unnecessary. The update only relaxes, and the result is already canonical: checking $d[i][j] \le d[i][l] + d[l][j]$ on the new matrix given it held on the old one, a path through a freed clock is $d[i][0] + \infty = \infty$; a freed row is ∞ throughout; a freed column becomes $d[i][0]$, and old closure already gave $d[i][0] \le d[i][l] + d[l][0]$. The reference row needs $d[0][l] + d[l][0] \ge 0$, which holds because a closed non-empty zone has no negative cycle.

The extrapolation closure. Extra_M closed unconditionally; a zone none of whose bounds exceeded its clock's maximum is unchanged and arrived closed, so a dirty flag skips the cubic step.

3.7 The static form

The same theorem applied once, at the root, to the top-level conjunction is a preprocessing step: split the conjuncts into atom-disjoint groups, solve each, conjoin. It costs one test and

it is strictly stronger on the shapes it catches, because no per-node bookkeeping is paid at all. It catches nothing whose independence only emerges during the search. Both forms are implemented; see Section 4 for the comparison.

4 What it buys

4.1 Selecting the search

The two searches are a command-line flag, `-version v1|v2`, so a comparison needs no rebuild:

`v1`	the search before this work: every optional optimisation off
`v2`	adds both decomposition forms and goal subsumption *(default)*

Two things about `v1` are worth stating plainly. It is a slower search, not a wrong one: the onset split points that the shifts introduce are a *soundness* fix, not an optimisation, and they are in both versions — without them $\mathrm{AG}(\mathrm{EF}_{[3,4]}p) \wedge \mathrm{AG}_{[4,5]}\neg p$ is reported satisfiable. And `v2` stands down when `-model` is given, because a witness for a decomposed node is the product of its components' automata and the extractor does not build products.

Measured end to end through the flag:

case	`v1` nodes	`v1` time	`v2` nodes	`v2` time	ratio
`wide` k=16	262 975	1.35 s	192	0.00 s	349×
`wide` k=20	4 195 583	31.38 s	240	0.00 s	**8309**×
`buried` k=14	66 181	0.34 s	731	0.01 s	43×
tower d=6	4 890	0.01 s	4 470	0.01 s	1×
tower d=8	21 025	0.05 s	19 349	0.05 s	1×
$\mathrm{AG}(\mathrm{EF}p) \wedge \mathrm{AG}(\mathrm{EF}q)$	352	0.00 s	92	0.00 s	2×

Run over the whole 5000-formula suite under both versions: **zero** disagreements, and zero mismatches against the by-construction expectations. The self-test passes 861/861 under `v1`, under `v2`, and under every individual flag combination.

4.2 The family it was built for

$\bigwedge_{j<k} \mathrm{EF}_{[2+j,\,4+j]}\, p_j$, per-node decomposition:

k	nodes off	nodes on	time off	time on
8	1 247	271	0.01 s	0.01 s
12	16 863	551	0.08 s	0.01 s
14	66 179	727	0.37 s	0.01 s
16	262 975	941	1.94 s	0.01 s
18	1 049 619	1 249	9.35 s	0.01 s
20	4 195 583	1 869	46.66 s	0.01 s

The exponential is gone. Node counts $271, 551, 727, 941, 1249, 1869$ are linear in k against a baseline that doubles, so the ratio is not a constant and grows without bound: 13× at $k = 12$, 244× at $k = 16$, 3478× at $k = 20$. Quoting a single speed-up figure would misrepresent it — the change is $O(2^k) \to O(k)$, and any number quoted is a statement about which k was measured.

The static form is stronger still on this shape, having no per-node cost: 1 247 → 96, 16 863 → 144, 262 975 → 192, and at $k = 20$ 4 195 583 → 240 — 46.66 s to below the timer, 11 884×.

4.3 The two forms compose

They see different things. The static form splits the root conjunction once and pays nothing per node; the dynamic form sees independence that only appears during the search. Neither subsumes the other, and running both takes the better of the two on every shape measured. Nodes, and time:

	plain	static	dynamic	both
`wide` k=16	262 975 / 1.91 s	192 / 0.00 s	941 / 0.01 s	**192**
`wide` k=20	4 195 583 / 46.6 s	240 / 0.00 s	1 869 / 0.01 s	**240**
`buried` k=10	4 437 / 0.02 s	4 437 / 0.02 s	403 / 0.01 s	**403**
`buried` k=14	66 181 / 0.45 s	66 181 / 0.40 s	731 / 0.01 s	**731**
`mixed` k=14	132 774 / 0.76 s	176 / 0.00 s	750 / 0.01 s	**176**

`buried` is $\mathrm{EF}_{[0,1]}(\bigwedge_j \mathrm{EF}_{[a_j,b_j]} p_j)$: the independence is real but sits under a modality, so the root is not a conjunction and the static form cannot see it at all — it returns the undecomposed node count exactly. `mixed` adds one shared conjunct, which the static split isolates and the dynamic one has to carry through every node.

4.4 The cost where it does not fire

Across the ten-family, 5000-formula suite, verdicts are **identical** to the undecomposed search. Wall-clock, $p95$ and max per family:

family	$p95$ off	$p95$ on	max off	max on
`wide`	42.8	**20.4**	69.3	**38.0**
`altern`	57.2	62.0	167.5	189.9
`persist`	13.1	15.5	15.6	18.0
`carry`	12.3	13.7	16.7	18.3
`release`	11.3	13.8	15.8	19.0
`boolmix`	14.3	14.2	17.6	17.9
`deadline`	14.2	14.0	18.2	15.8

All figures in milliseconds. The medians across every family sit near 8–10 ms and are dominated by process start-up, so only $p95$ and max carry signal.

The honest reading: a large win on the one family that decomposes, and a 5–10% tax everywhere else, paid for running a test that answers "no". The bitmasks and the no-split memo took that tax down from about 30%; what remains is building the item vector at nodes whose shape has not been seen before.

4.5 What these numbers do and do not show

They *do* show that the 2^k in Section 1 is an artefact of representation and that removing it is sound on every instance measured (5000 formulas, plus a 700-case soundness sweep over $\mathrm{AG}(\mathrm{EF}_{[a,b]} p) \wedge \mathrm{AG}_{[c,d]} \neg p$, plus the 861-check self-test).

They *do not* show that this helps on formulas people write. `wide` is a family from our own suite, chosen to be a conjunction of independent obligations, which is exactly the shape the technique targets. The fair claim is about the mechanism, not about a speed-up on a workload: conjunctions of independent properties are common in practice — a safety property and a liveness property over different signals — but this suite is not evidence for how common.

A second caveat: the static form's numbers are on top-level conjunctions, and splitting a top-level conjunction is close to running the tool twice. It should be presented as preprocessing, not as a search technique.

5 What does not work, and why

This section exists so nobody spends a second week on these. Each was implemented and measured, not reasoned about.

5.1 Zone subsumption

Realisability is monotone under narrowing, so an accepted verdict transfers to every sub-zone and a rejected one to every super-zone — the passed-list inclusion test of a reachability tool. Implemented, it gave 2% fewer nodes and cost slightly more than it saved.

The reason is structural: the several zones a discrete shape carries are the *disjoint* cells that the delay rule produces, and disjoint zones are never subsets of one another. Inclusion has nothing to fire on.

5.2 Coarser abstraction (Extra$_{LU}$)

Strictly coarser than Extra$_M$ and already available in the DBM library. It cannot help as the construction stands: the split points a clock is tested against are used as *equalities* by the delay rule, so a clock's lower and upper bound sets coincide and Extra$_{LU}$ collapses to Extra$_M$. It would need the splitting reworked to be one-sided first.

5.3 Forced stops

If ψ is already a goal and the whole zone lies inside the window, the stop branch is available at every valuation and satisfies the until outright, so the deferral branch looks redundant. It is not. Two satisfiable formulas turn unsatisfiable:

$$\mathrm{AG}\,\mathrm{AF}_{[0,3]}q \wedge \mathrm{AG}(q \to \mathrm{AG}_{(0,1]}\neg q), \qquad \mathrm{AG}(p \wedge \mathrm{A}(q\,\mathbf{U}_{[1,3]}\,r)) \wedge \mathrm{EG}(r \to \mathrm{EG}\,r).$$

Incomplete rather than unsound, so it never lies about UNSAT — and the node counts before the check were tempting (21 025 → 7 326 on a depth-8 tower), which is the whole reason to record it here.

5.4 Collapsing the pursuit record

Of the fields a pursuit carries, `is_until` and `x_mu` are functions of the modality, and `rolled` is recoverable from the deadline (which is b before the first witness and $b - a$ after; when $a = 0$ the two coincide and so do the guards). All three are therefore *functionally determined*, and dropping them from the key merges nothing — a field determined by μ takes the same value at every node carrying μ. Only `part` is genuinely recoverable from data that varies, and only when the zone lies wholly in one part of the window; canonicalising it there is worth a few per cent on timed releases and nothing elsewhere.

5.5 Goal subsumption

$\mathrm{Q}(\varphi\,\mathbf{R}_I\,\psi)$ with $0 \in I$ asserts ψ at this configuration, and a disjunction is implied by any disjunct already present, so both consequences can be dropped from Γ. Worth 8% on towers and 36% on shapes that actually carry the redundancy; roughly neutral on the suite.

One trap: the first version dropped *literals* too, and $\mathrm{EG}(\mathrm{AF}\,p \wedge \mathrm{AG}\,\neg p)$ came back satisfiable. Closure is decided on Γ_{prop}, so a literal removed here is a literal the contradiction test cannot see, and the release only restores it at the next expansion, by which time the complementary literal may have been consumed.

5.6 Treating $\mathrm{AG}\,\psi$ as context

$\mathrm{AG}\,\psi \equiv \psi \wedge \mathrm{AX}\,\mathrm{AG}\,\psi$, and the calculus already carries "holds at every successor". Encoding it that way removes a live pursuit per nesting level. It is a large win where such modalities sit *side by side* (106 679 → 14 809 nodes on six parallel $\mathrm{AG}(\mathrm{EF}\cdots)$) and a growing loss where one sits *inside* another (-50% at tower depth 9), because an AG under an until is re-demanded through the context at every onset that until relaunches. Restricting it to modalities under no enclosing until keeps the wins and caps the loss at 8%. Left behind a flag, default off; the regression is real and unbounded in depth.

5.7 Single-polarity atoms as non-linking

The partition joins two items whenever they share an atom, so a conjunction of k obligations that all mention one shared enabling proposition never decomposes — the shape `almost` in the adversarial suite, and the one that most obviously ought to yield.

The argument for splitting anyway: the input is in negation normal form, so an atom occurring under a single polarity cannot be the subject of a disagreement. Give the product the union of the two labellings for an atom that only ever occurs positively and the intersection for one that only occurs negatively; each component's formulas should then survive by monotonicity, a positive literal being monotone upwards in the states where it holds and a negative one downwards, with the polarity uniform so there is no other constraint to violate. Only an atom appearing both ways would link, and data variables would stay exempt, a constraint over one being monotone in neither direction.

Implemented, the polarity analysis is correct — on $\mathrm{AG}_{[0,4]}\mathrm{E}(p\,\mathbf{U}_{[1,2]}q)\wedge\mathrm{AG}_{[0,3]}\neg q$ it classifies q as both-polarity and p as positive-only, which is what the rule intends — and five self-test checks fail, including that formula, which is unsatisfiable and comes back satisfiable. So the monotonicity argument has a gap. A plausible place to look is that a component is decided against a *node*, not against the root formula, and an AND-sibling of that node carries constraints that the component's own atoms do not mention; but that is a conjecture, not a diagnosis.

Recorded rather than fixed: the idea is worth more thought, the reasoning above is where to resume, and the failing formula is the test to resume against.

5.8 Conflict-driven anything

See Section 1: zero rejected and zero closed nodes on the hardest instance. There are no conflicts to learn from.

6 Limits

It removes one exponential, not the exponential. Theorem 2.2 attacks the product of *independent* obligations. The nesting towers of Section 1 are not independent — each level's AG sits inside the previous level's EF and they share atoms — and their growth is untouched. Six separate attempts on that growth are recorded in Section 5; none moved it. The remaining growth is believed to be the genuine $2^{O(n)}$ of the bound.

The test can only see syntactic independence. Two components sharing a symbol are never split even when the symbol is used in ways that do not interact. The symbols counted are atomic propositions and data variables (Section 3); a semantic independence test would catch more and cost more, and nothing here attempts one. Note which way the error runs: a

missed split costs time, an unsound one costs correctness, and the twelve self-test failures on $\{x \leq 5\} \wedge \{x \geq 10\}$ are what happens when the symbol set is too small.

The suite is our own. The family that decomposes is one we wrote. See the caveat in Section 4: the claim to make is about the mechanism, not about a speed-up on a workload.

Witness extraction is disabled under decomposition. A witness for a decomposed node is the product of the components' automata, and the extractor does not build it. With `-model` the static form declines to decompose; the per-node form has not been reconciled with witness extraction at all, which is the first thing to fix if either is turned on by default.

The overhead is not zero. 5–10% on inputs that never decompose (Section 4). Turning either form on by default needs either a cheaper negative test or evidence that real inputs decompose often enough to pay for it.